



SOFTWARE ARCHITECTURE DOCUMENT

# FixPulseHQ Bug Tracker

Complete System Architecture & Technical Reference

Node.js 18+

Express 4

React 18 (UMD)

PostgreSQL · Neon

Redis · ioredis

JWT Auth

RBAC

PM2 Cluster

Worker Threads

Google Sheets API

Razorpay

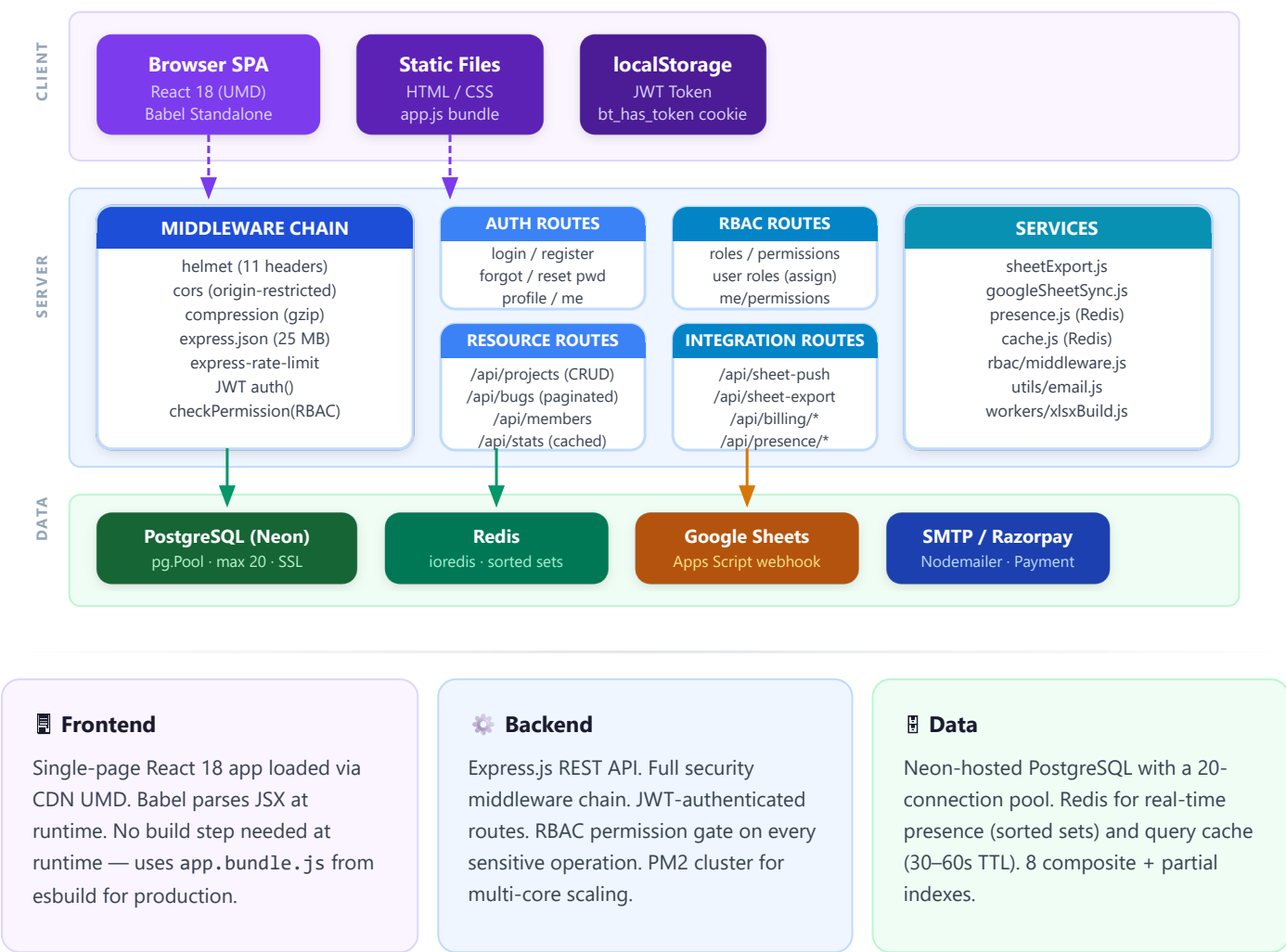
Nodemailer

**Organization:** Palletnow Technologies · **Product:** FixPulseHQ v2.0.0

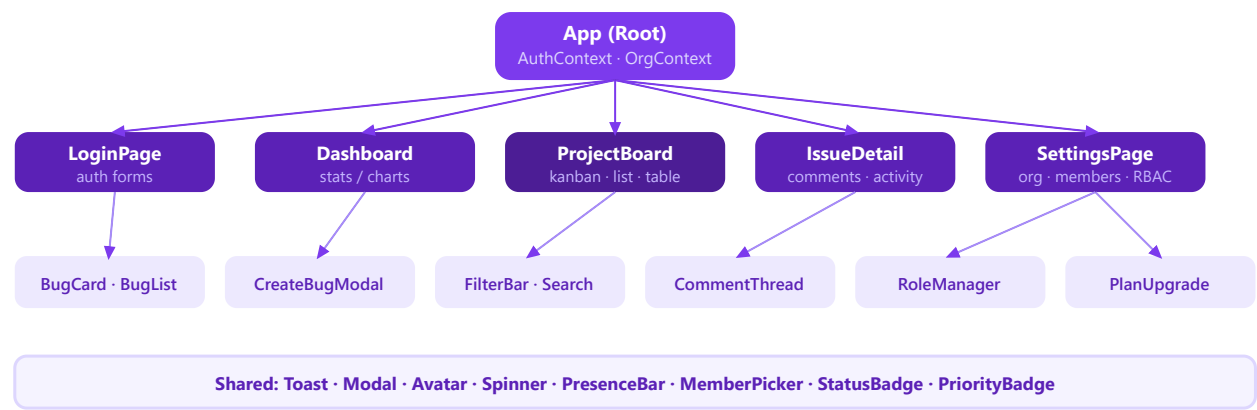
**Document Type:** Architecture & System Design · **Classification:** Internal / Confidential

**Prepared by:** Engineering Team · **Date:** April 2026

FixPulseHQ is a **multi-tenant SaaS bug tracking platform** built on a single Node.js process (clustered to all CPU cores via PM2). The system follows a classical **3-tier architecture**: browser client → Express API server → PostgreSQL database, augmented with Redis for horizontal-scaling concerns (presence and query caching) and a Worker Thread pool for CPU-intensive XLSX generation.



The frontend is a **single-page application (SPA)** built entirely in React 18 running inside a single HTML shell. There is no frontend framework router dependency — routing is handled by React state. Babel Standalone compiles JSX at runtime in development; esbuild produces `app.bundle.js` for production.



State Management

React Context (no Redux). Two global contexts:

**AuthContext** — holds JWT token, current user object, org payload. Persists in `localStorage`. Token is sent as `Authorization: Bearer <token>` on every API call.

**OrgContext** — holds org settings including plan code, data source config, and Apps Script URL.

Build Pipeline

**Development:** Babel Standalone loaded from CDN parses JSX on the fly in the browser. No local build step. Hot reload via nodemon.

**Production:** `npm run build:app` runs esbuild, bundles `public/app.js` → `public/app.bundle.js` (IIFE format). Static assets served with `maxAge: 1h` and ETags.

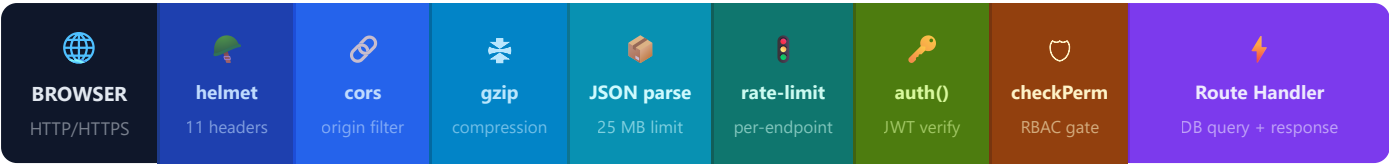
Routing Strategy

No React Router. App checks `bt_has_token=1` cookie to decide which shell to serve. The Express catch-all redirects unknown paths to `index.html` or `app-shell.html`. Pages are React state variables toggled by sidebar nav.

API Communication

All API calls are plain `fetch()` with JSON. The base path is the same origin. Attachments are encoded as base64 data URLs inline in JSON. Large responses (bugs list) are paginated at 20 items per page.

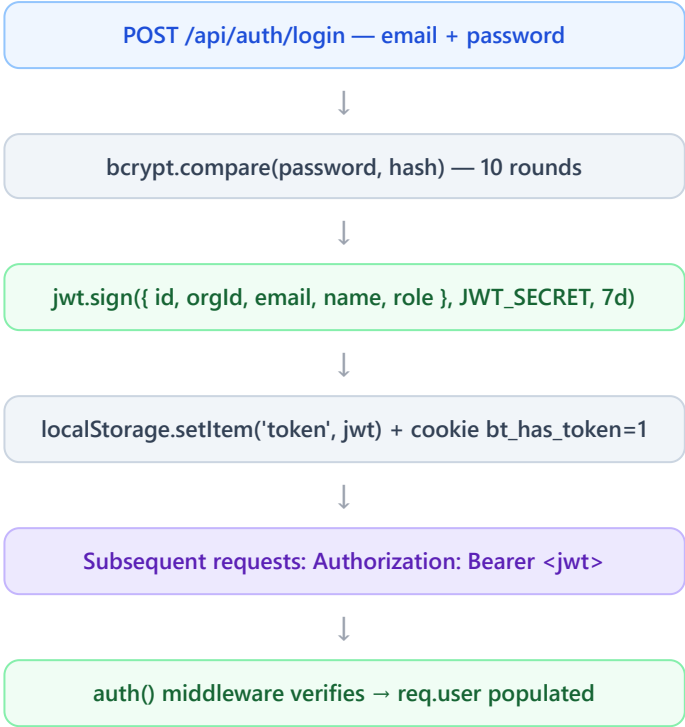
Every HTTP request passes through a strict ordered middleware chain before reaching a route handler. The chain is designed so that **failures at earlier layers abort the request early**, reducing DB load and improving security.



Security Middleware Detail

| Middleware          | What it enforces                                                                                                                      |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| helmet              | X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Strict-Transport-Security, Referrer-Policy, X-DNS-Prefetch-Control + 6 more   |
| cors                | Only allows origins listed in CORS_ORIGIN env var. Warns at startup in production if unset.                                           |
| rate-limit (auth)   | 30 requests per 15-minute window on /api/auth/login and /api/auth/register. Brute-force protection.                                   |
| rate-limit (export) | 10 req/min on /api/sheet-export and /api/sheet-push. Prevents DoS via CPU-heavy XLSX build.                                           |
| auth()              | Verifies Authorization: Bearer <JWT>. Attaches req.user = { id, orgId, email, name, role }. 401 if missing or expired.                |
| checkPermission     | Resolves user's effective permissions from RBAC tables. Supports project-level scoping via dynamic projectId resolver. 403 if denied. |

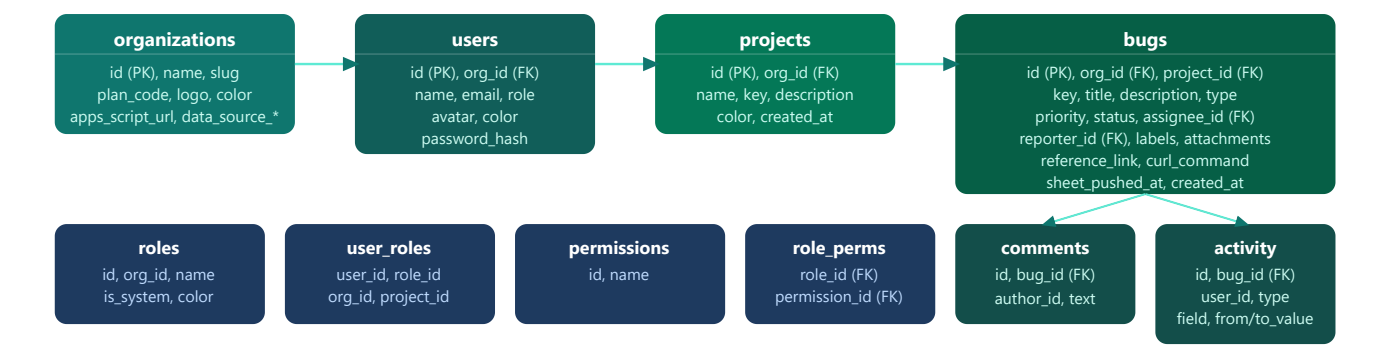
JWT Token Flow



Password Security

**bcryptjs** with cost factor 10 (~100ms hash time). Passwords never stored in plain text. Password reset uses a randomUUID() token stored in password\_reset\_tokens table with 1-hour expiry.

PostgreSQL hosted on **Neon** (serverless, edge-cached). Connection managed by `pg.Pool` (max 20 connections, 30s idle timeout, 5s connection timeout). All tables are scoped by `org_id` for multi-tenancy.



Performance Indexes

| Index Name            | Type & Purpose                                          |
|-----------------------|---------------------------------------------------------|
| idx_bugs_org_project  | Composite — accelerates WHERE org_id=X AND project_id=Y |
| idx_bugs_assignee     | Partial (non-null) — filter bugs by assignee            |
| idx_bugs_status       | Composite — status filters + stats queries              |
| idx_bugs_created_at   | DESC — default sort order                               |
| idx_bugs_sheet_pushed | Partial (IS NULL) — incremental sheet push              |
| idx_activity_bug_id   | Single — activity join on bug detail                    |
| idx_comments_bug_id   | Single — comments join on bug detail                    |

Multi-Tenancy

Every table except roles and permissions carries an `org_id` column. All API routes filter by `req.user.orgId` which comes from the JWT token, preventing any cross-org data access.

Organizations have isolated: users, projects, bugs, roles, settings, data sources, and billing orders.

Connection Pool

**max: 20** — up to 20 concurrent DB connections per Node.js process. With PM2 cluster (N cores), total pool size = 20 × N connections. Size N's Neon plan accordingly.

**idleTimeoutMillis: 30s** — returns idle connections to pool.

**connectionTimeoutMillis: 5s** — fails fast if pool exhausted.

Pool error events are caught and logged — prevents Neon ECONNRESET from crashing the process.

FixPulseHQ implements a **flexible, two-layer RBAC system**. Users can have organisation-level roles (global) and project-level roles (scoped). The system merges permissions from both layers for any given request context.

| 11 PERMISSIONS       | Admin | Proj Mgr | Developer | QA/Tester | Viewer | Custom Roles                                                          |
|----------------------|-------|----------|-----------|-----------|--------|-----------------------------------------------------------------------|
| • CREATE_ISSUE       | ✓     | ✓        | ✓         | ✓         | —      | Any combination of permissions per org + project-level role overrides |
| • EDIT_ISSUE         | ✓     | ✓        | ✓         | ✓         | —      |                                                                       |
| • DELETE_ISSUE       | ✓     | ✓        | ✓         | ✓         | —      |                                                                       |
| • ASSIGN_ISSUE       | ✓     | ✓        | ✓         | ✓         | —      |                                                                       |
| • CHANGE_STATUS      | ✓     | ✓        | ✓         | ✓         | —      |                                                                       |
| • COMMENT            | ✓     | ✓        | ✓         | ✓         | —      |                                                                       |
| • VIEW_ISSUE         | ✓     | ✓        | ✓         | ✓         | ✓      |                                                                       |
| • VIEW_REPORTS       | ✓     | ✓        | ✓         | ✓         | ✓      |                                                                       |
| • MANAGE_PROJECT     | ✓     | ✓        | —         | —         | —      |                                                                       |
| • MANAGE_USERS       | ✓     | —        | —         | —         | —      |                                                                       |
| • CONFIGURE_WORKFLOW | ✓     | —        | —         | —         | —      |                                                                       |

### Permission Resolution Algorithm

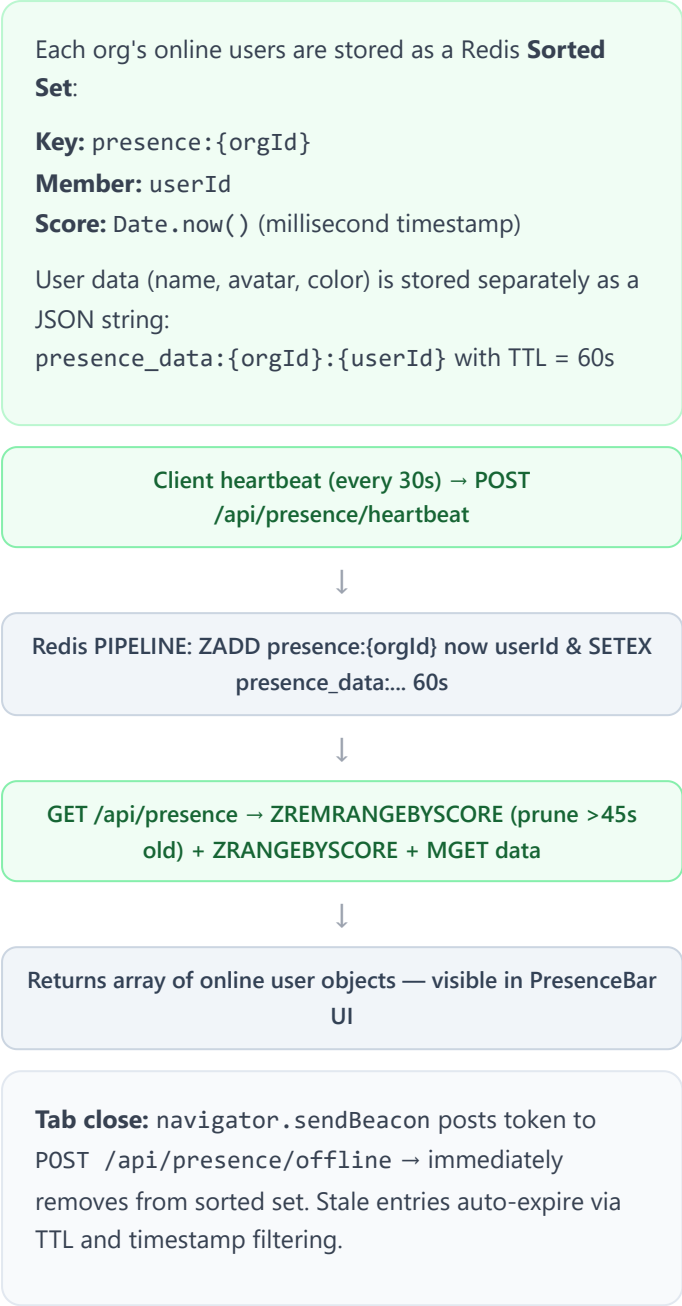
1. Query user\_roles JOIN role\_permissions JOIN permissions where user\_id=\$1, org\_id=\$2, and project\_id IS NULL OR project\_id=\$3.
2. If rows found → return merged Set of permission names.
3. If no user\_roles rows at all → fall back to legacy users.role column → map to LEGACY\_ROLE\_PERMISSIONS.

### Project-Level Scoping

A user can be a Viewer org-wide but a Developer on one specific project. The checkPermission middleware accepts a dynamic projectId resolver function — for bugs, it looks up the bug's project before checking permissions. The SQL query merges org-level + project-level permissions automatically.

Redis enables two features that require **shared state across multiple Node.js processes**: real-time user presence and query result caching. Both degrade gracefully to in-process fallbacks when REDIS\_URL is not set.

Presence System (Sorted Sets)



Query Cache

| Cache Key                              | TTL | Invalidated by                            |
|----------------------------------------|-----|-------------------------------------------|
| <code>members:{orgId}</code>           | 60s | POST/PUT/DELETE <code>/api/members</code> |
| <code>stats:{orgId}:all</code>         | 30s | Bug create / update / delete              |
| <code>stats:{orgId}:{projectId}</code> | 30s | Bug create / update in project            |

Cache hit: `cache.get(key)` → returns parsed JSON, skips DB entirely.

Cache miss: DB query runs → result stored with SETEX key TTL JSON.

Cache write failures are silently swallowed — Redis being down never breaks a request.

In-Process Fallback

When REDIS\_URL is absent or Redis is unreachable:

- Presence** → falls back to `Map<orgId, Map<userId, entry>>` in memory (single-instance only)
- Cache** → all `cache.get/set/del` calls are no-ops; every request hits the DB

The server logs ⓘ REDIS\_URL not set – using in-process fallback at startup.

Redis Connection Config

`maxRetriesPerRequest: 1` — fails fast, doesn't queue

`enableOfflineQueue: false` — drops commands when disconnected

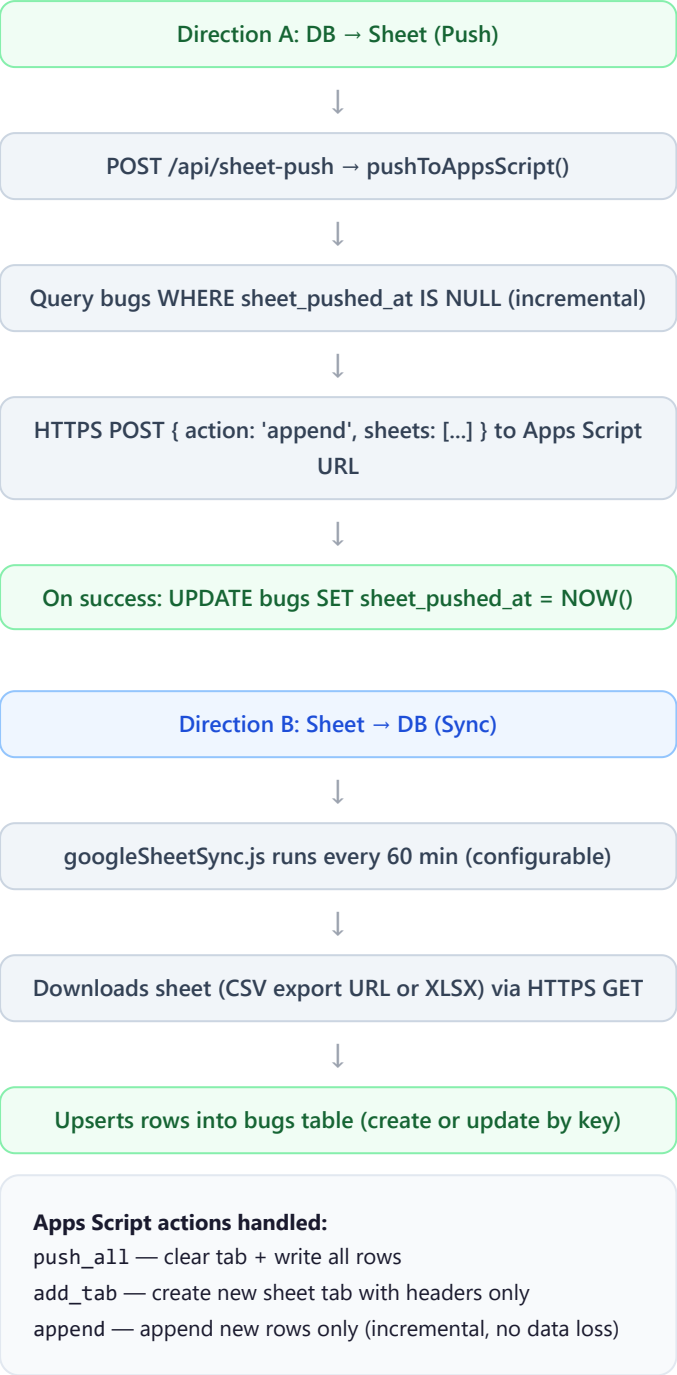
`connectTimeout: 5000ms` — startup fail-fast

`lazyConnect: true` — explicit `redis.connect()` in boot sequence

GOOGLE SHEETS VIA APPS SCRIPT

The app integrates with Google Sheets using a **deployed Apps Script Web App** as a serverless HTTP endpoint. No Google credentials are stored — the script is publicly deployed ("Anyone" access).

Two-way sheet integration



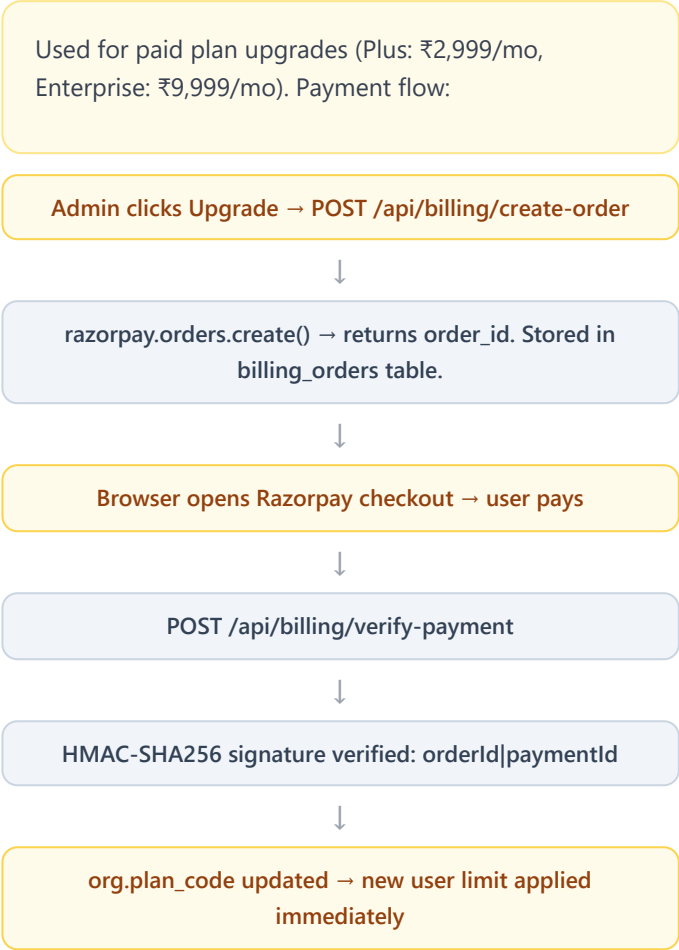
SMTP EMAIL (NODEMAILER)

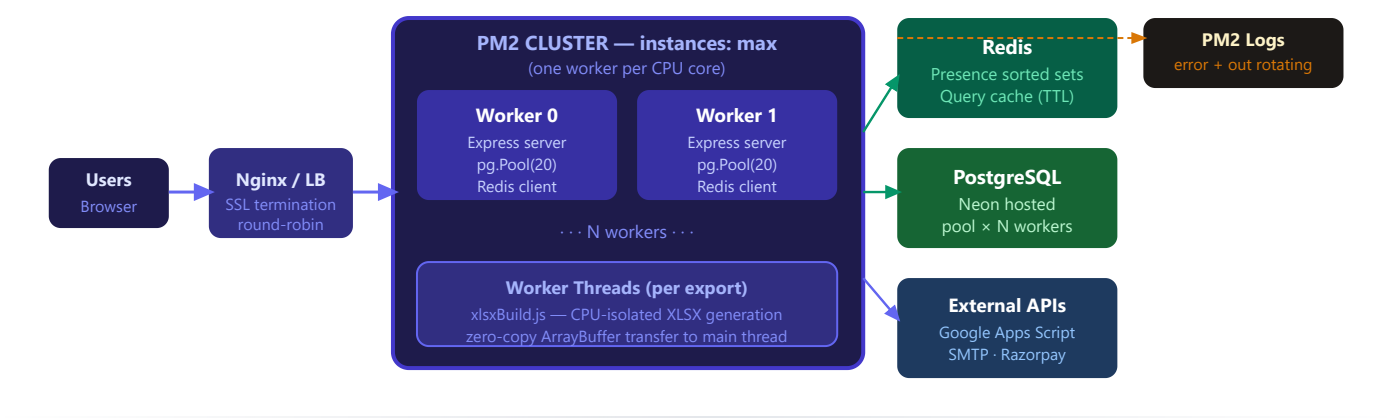
Nodemailer sends transactional emails for two events. The transporter is lazily initialised on first send and reused for all subsequent sends.

All sends are **non-blocking** — the API response is returned immediately and email delivery is a fire-and-forget `.catch()` chain. SMTP failures never break a request.

| Event            | Trigger                                                                                               |
|------------------|-------------------------------------------------------------------------------------------------------|
| Issue Assignment | POST /api/bugs — when assigneeId is set. Sends assignee name, issue key, type, priority, description. |
| Password Reset   | POST /api/auth/forgot-password. Sends a randomUUID() token link valid for 1 hour.                     |

RAZORPAY PAYMENT GATEWAY





Graceful Shutdown

| Signal        | Behavior                                                                                                                         |
|---------------|----------------------------------------------------------------------------------------------------------------------------------|
| SIGTERM       | Stops accepting new connections. Waits for in-flight requests to complete. Exits cleanly. (Container stop / Kubernetes eviction) |
| SIGINT        | Same as SIGTERM. Triggered by Ctrl+C in terminal.                                                                                |
| Timeout (15s) | If requests don't drain in 15s, process.exit(1) is forced. .unref() ensures the timer doesn't prevent normal exit.               |

HTTP Tuning

`server.keepAliveTimeout = 65_000ms` — exceeds AWS ALB 60s idle timeout. Prevents 502 errors from keep-alive connection races.

`server.headersTimeout = 70_000ms` — must be > `keepAliveTimeout`. Prevents node from sending headers on a half-closed connection.

PM2 Configuration Highlights

```
instances: 'max' — PM2 detects CPU count at runtime
exec_mode: 'cluster' — Node cluster API, shared port
max_memory_restart: '512M' — auto-restart leaky workers
kill_timeout: 15000 — matches graceful shutdown window
autorestart: true + max_restarts: 10
merge_logs: true — consolidated log file
Zero-downtime reload: pm2 reload
ecosystem.config.js
```

Capacity Estimates

| Setup                       | Concurrent Users |
|-----------------------------|------------------|
| 1 core, no Redis            | ~50–200          |
| 4 cores + Redis             | ~500–2,000       |
| 8 cores + Redis + Neon Pro  | ~2,000–8,000     |
| 8 cores + Redis + PgBouncer | 10,000+          |